Post-training for Persona and Capability

Agastya Sridharan, Sam Huang, Andrew Wu

Contents

1 Introduction **2**

2 SFT **2**
2.1 Data generation and filtering . . . 2
2.2 Training . . . 2
2.3 Adding math data . . . 2
2.4 Math results . . . 2
2.5 Sheldonness, with LLM-as-a-judge . . . 3
2.6 Sheldonness, with a logistic regression . . . 3

3 RLAIF **3**
3.1 Introduction . . . 3
3.2 Auditing and another SFT . . . 3
3.3 RLAIF Prompt set . . . 4
3.4 The judge and the reward formula . . . 4
3.5 Training and Evaluation . . . 4

4 RLVR **4**
4.1 Algorithm . . . 4
4.2 Training Runs . . . 5

1 Introduction

We aimed to post-train Qwen2.5-3B-Instruct to speak in the voice of Sheldon Cooper, and for it to improve at math. Sheldon is a fictional character from the TV show *The Big Bang Theory*, a theoretical physicist. Here is how Sheldon speaks: all quotations are taken from the show.

- "Penny, while I subscribe to the many-worlds theory which posits the existence of an infinite number of Sheldons in an infinite number of universes—I assure you that in none of them am I dancing[.]"
- "That's the rankest psychologism, and was conclusively revealed as hogwash by Gottlob Frege in the 1890s!"

We chose the GSM8k dataset to evaluate math performance. It contains questions like: "Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?"

2 SFT

2.1 Data generation and filtering

We used deepseek-v4-flash-0731 to generate Sheldon-esque text. It played both the user and Sheldon. Roughly 20000 examples were generated total, including both math and non-math chat examples.

Raw DeepSeek data was overfit on certain Sheldon-esque qualities. In particular, roughly 21% of rows contained "Bazinga!," a classic Sheldon catchphrase, roughly 44% of rows mentioned Leonard, another character, and 1111 rows began with "I am about to make [...]."

We therefore filtered the dataset before training. For our first SFT, we dropped the math rows, along with examples where the assistant response was over 600 words, extremely short, or contained Chinese or Japanese text (a Deepseek artifact). We also capped repetitive phrases: no five-word opener could exceed 1% of the filtered dataset, and "Bazinga," "about to make a joke," and "scale of one to ten" were capped at 10%, 2%, and 1%. This left 11910 chat examples.

Before training, we set aside a held-out persona set (system prompt and first user message only, no Sheldon response) so that every later model could be compared on the same unseen inputs, and reserved 2% of the training data to track validation loss.

2.2 Training

All SFT runs started from Qwen2.5-3B-Instruct and used LoRA. We used rank 32, $\alpha = 64$, and dropout 0.05, applying LoRA to the query, key, value, output, gate, up, and down projections.

We trained for 2 epochs using AdamW with learning rate 10^{-4}, a cosine learning-rate schedule, and 3% warmup. The effective batch size was 64 sequences (16 sequences per batch with 4 gradient-accumulation steps), with a maximum sequence length of 2048 tokens. Training used bfloat16.

We used assistant-only loss (system and user tokens did not contribute), and saved and validated roughly 10 checkpoints per epoch.

2.3 Adding math data

At first, we SFT'ed only on the 11910 non-math chat examples. We call this model **v2**. This produced the persona but substantially damaged math performance (likely catastrophic forgetting).

We therefore tried two ways of adding math data. For **v3a**, we took the math rows of the original Sheldon dataset whose boxed answer matched the reference, applied the same filtering as for chat, and added the resulting 2036 verified-correct examples to the v2 set.

For **v3b**, we additionally generated structured Sheldon-style rewrites of 4800 GSM8k training solutions (checked against the test set for near-duplicates): one arithmetic step per line, a Sheldon remark of at most 60 words, a short quip, and the answer in a box. A deterministic verifier checked that the boxed answer and intermediate arithmetic matched the reference, that the GSM8k reasoning steps were present, and that there was no prohibited formatting or generic chatbot language; 15 examples whose GSM8k reference the generator flagged as wrong were dropped. After filtering and reserving 300 for validation, 4485 generated examples were added.

Thus, our three main SFT variants were:

- **v2**: 11910 Sheldon chat examples and no math data;
- **v3a**: v2 plus 2036 verified-correct Sheldon math examples;
- **v3b**: v3a plus 4485 structured Sheldon rewrites of GSM8k training examples.

2.4 Math results

We evaluated math performance zero-shot on the GSM8k test set using greedy decoding and numerical matching of the boxed final answer.

Model	GSM8k accuracy	Sheldon references in math
Base Qwen	86.7%	0.8%
v2	50.8%	65.5%
v3a	67.1%	98.9%
v3b	63.2%	10.3%

Chat-only SFT therefore caused a large capability regression: GSM8k accuracy fell from 86.7% to 50.8%. Adding verified-correct math data recovered much of this loss, with v3a reaching 67.1%. However, v3a almost always mixed the persona into its math solutions (98.9% of answers contained Sheldon references). The structured v3b data cut this to 10.3%, with a boxed answer 100% of the time and an average of 3.5 equation lines per answer, at a slightly lower 63.2%. Across repeated and intermediate runs the math-data variants stayed in the 63–67% range, so more formatted examples did not remove the remaining reasoning regression.

All three SFT runs showed the same odd dynamic: in roughly the first 10% of training, GSM8k accuracy collapsed to 25–35% before recovering within one or two checkpoints. Only about 14% of the final models' wrong answers contained a simple arithmetic error, so arithmetic mistakes do not explain the remaining gap.

2.5 Sheldonness, with LLM-as-a-judge

We picked Claude Sonnet 4.6 at temperature 0 and created a "Sheldonness" rubric with six axes (pedantic precision, superiority as fact, rules and routines, formal register, tangent or reference, social obliviousness), each scored 0 to 2. Below, "Gold" is DeepSeek's original response, Base is non-SFT'ed Qwen, and v3b is SFT'ed Qwen.

Response type	Mean Total
Gold	8.00
v3b	6.50
Base	0.04

Our judge evaluated the SFT'ed model as quite Sheldon-like, though less so than DeepSeek's original responses.

2.6 Sheldonness, with a logistic regression

We also trained a logistic regression classifier to distinguish DeepSeek's Sheldon responses from base Qwen responses, on roughly 84000 character and 56000 word n-grams (from "excuse me, which is not," character n-grams include 'ex,' 'use,' and 'cuse,' and word n-grams "excuse," "which," and "excuse me") with log-damped, inverse-document-frequency-weighted counts (so repeating a phrase had diminishing returns and common fragments counted less). Names and catchphrases were removed from the input, since otherwise the classifier relied on those superficial features. We then evaluated it on Gold, Base, and v3b responses to held-out prompts:

Gold-vs-base accuracy	Gold	Base	v3b
0.998	0.798	0.238	0.818

The first number is the classifier's Gold-vs-base accuracy. The remaining numbers are its average predicted probabilities of belonging to the Sheldon-response class. Under this measure, v3b's responses were evaluated as slightly more Sheldon-like than DeepSeek's original responses.

3 RLAIF

3.1 Introduction

RLAIF essentially works as follows: a model generates answers to prompts, and an AI judge scores each answer with a number, the reward. The model is updated to make high-reward answers more likely.

We used the algorithm **Group Relative Policy Optimization** (GRPO). For each prompt, we did the following:

1. Sample $G = 8$ different responses from the model (with temperature 0.8);
2. Score each response: $r_1, \ldots, r_8$;
3. Compute the **advantage** of each response,
$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_8)}{\text{std}(r_1, \ldots, r_8)};$$
4. Update the model: raise the probability of the tokens of responses with $A_i > 0$ in proportion to A_i, and lower it for responses with $A_i < 0$ in proportion to $|A_i|$.

We used clipping ($\epsilon = 0.2$) and KL penalty ($\beta = 0.04$) as regularization, to ensure the updates were not too large. We used Hugging Face's GRPOTrainer and trained with LoRA.

3.2 Auditing and another SFT

First, we audited our SFT'ed model's (v3b's) responses to the 502 held-out test set prompts. We found the following:

- 97% of replies started with one of six templates;
- 22% of replies hit the 400-token cap;
- 18% of replies had the phrase "I am about to make a joke.";
- We also found instances of fabricated corrections (claiming a user had made a typo when there was no such typo), task failures on constrained prompts (answers that went over a user-imposed word limit), and self-contradictions.

GRPO can only reward responses that get produced: if v3b almost never produces a reply that doesn't start with one of those templates, we can almost never reward that. So before RL, we did an SFT touch-up on a new set: (1) 2999 non-math rows of the original SFT data with canon errors (e.g. "Tuesday = Thai food"; in the show it is cheeseburger night), stock phrases, and repeated sentences dropped; (2) 800 of its math rows; and (3) 813 short replies written by gpt-4.1-mini and Claude Haiku, because v3b was always verbose.

On the 502 held-out prompts, the touch-up'ed model (v4) started 66% of replies with one of the six templates and hit the 400-token cap in 19%, though 28% now contained "I am about to make a joke." Claude Sonnet 4.6 as judge preferred v4 to v3b.

3.3 RLAIF Prompt set

We built an RL prompt set with 5714 prompts.

- 3299 persona prompts: “Hey, what do you actually do for fun? Like, you can’t lift, so ... solitaire?”
- 1000 short prompts: “which marvel villain is the most sympathetic”
- 500 format-constraint prompts: “how many minutes in a week use at most 80 words”
- 400 two-turn prompts, with a second user turn like “okay but why so aggressive about the rivers”
- 300 “pick a side” prompts: “Settle this for me: which one?”
- 120 AI-identity prompts: “Are you Qwen?”
- 95 “sensitive” but legitimate prompts: “Any practical tips befor I call the non-emergency line?”

3.4 The judge and the reward formula

We used gpt-5.6-luna as the AI judge. For each of the 8 responses i to a prompt, the reward used the following terms:

- G_i measured task adherence. The judge lists every explicit request and constraint and averages adherence to them (taskscore), then flags a refusal (refused = 0, else 1), a wrong main answer or harmful information (worseoff = 0.25, else 1), and a self-contradiction (contradiction = 0.5, else 1); $G_i = \text{taskscore} \cdot \text{refused} \cdot \text{worseoff} \cdot \text{contradiction}$.
- P_i was the persona win rate: how often a response was judged more Sheldon-esque than the other responses v4 wrote for the same prompt. Responses were shuffled into a circle and each pair of adjacent responses was given to the judge twice, once in each order, to avoid any bias of “which one did the judge see first?”; the judge, equipped with a Sheldon style guide, decided which response adhered to it better. If response i won a proportion p of its comparisons, P_i was set to $2p$.
- F_i was a bonus for good form. Prompts were classified as phatic, one-line, brief, long, default, or math, and a response earned 0.25 each for staying within the class’s word budget (for example, 130 words for prompts that say “briefly” or “tl;dr”), for keeping the preamble (the words before v4 actually engages with the prompt) short, and for answering a phatic prompt like “good morning!” very briefly.
- RulePenalty_i aggregated 17 deterministic checks: false claims, fence-sitting when asked to pick a side, assistant-like tool voice (“feel free to,” “hope this helps”), out-of-character behavior (a sincere apology, being a sports fan), and so on.
- FalseClaim_i was 1 if v4 made a false claim about the user, e.g. “correcting” a word the user never wrote.
- BatchTax_i taxed responses that leaned on stock phrases, repeated openers and closers, verbatim long sentences, or cast names over-used across the last 512 responses.

The reward was calculated as follows:

$$R_i = G_i \cdot (2P_i + F_i) - \text{RulePenalty}_i - 0.5 \cdot \text{FalseClaim}_i - \text{BatchTax}_i.$$

3.5 Training and Evaluation

Each training step took 16 prompts, generated 8 responses each, computed rewards, and took one gradient step on the LoRA adapter; checkpoints were saved every 25 steps. We did two runs: the first was stopped at step 90, the second at step 109 (a 4-hour cap). The second model, grpo-v2, was compared with its predecessor v4 by gpt-5.6-luna as follows:

- 200 prompts never used in training, with replies from both v4 and grpo-v2;
- the judge was asked which reply was more Sheldon-esque, under the same criteria used for P_i;
- each pair was judged twice, once in each order.

The result was 50%: the judge did not favor either model, so the RLAIF result was null. Math held (v3b 63.7% on GSM8k test, grpo-v2 64.0%). The measurable gain came from the pre-RLAIF touch-up, which reduced template openers and canon errors. We think the learning rate was too low for the number of steps we could afford; with a higher one we suspect the result would not have been null.

4 RLVR

4.1 Algorithm

For the core algorithm we used CISPO (from the MiniMax-M1 paper): in both the MiniMax and the ScaleRL experiments it significantly outperforms GRPO and its variants (DAPO, GSPO, etc.), and MiniMax’s experiments also use a Qwen2.5 base model, just the 32B version. For loss aggregation we used DAPO-style token-level loss (gradients averaged over all tokens within a group) rather than GRPO-style sequence-level loss, under which a token of a rollout of length L_1 is weighted $\frac{L_2}{L_1}$ as strongly as a token of a sibling of length L_2; ScaleRL supports this too. For std normalization we used group-level rather than batch-level normalization, mostly to upweight “extreme” groups that are almost all 0s or almost all 1s. Other settings: the CISPO clip was $\epsilon_{\max} = 5$ with no lower clip and no KL term; rollouts were sampled at temperature 1 with a 2048-token cap, and truncated rollouts and zero-variance groups were masked out of both the loss and the group statistics; the learning rate followed a warmup-stable-decay schedule.

Our rollout infra was vLLM, while our training infra was HF’s TRL, so we used an FP32 LM head to avoid small numerical differences in the kernels affecting training. The global batch size was 256, with group size 16 and 16 groups per batch. Our verifier was just if the boxed answer was numerically equal to the gold answer after parsing the LaTeX.

4.2 Training Runs

This table summarizes our run results, which are explained in more detail below. MATH-500 is run at temperature 0.6, persona + GSM8K is at greedy.

	Math			Persona eval (502 held-out prompts)		
Run	MATH-500	MATH-500 L3–5	GSM8K	Judge win vs. RLAIF	Rule penalty ↓	Mean words
Base (Qwen2.5-3B-Instruct)	66.6	59.0	86.1	–	–	–
RLAIF (seed)	29.6	21.0	63.6	0.500	0.87	213
Run 1	63.5	55.2	82.0	0.545	1.02	246
Run 2	63.6	55.3	82.0	–	–	–
Run 3	65.1	56.9	83.0	0.495	1.20	255
Run 4	64.8	56.9	83.0	0.485	0.15	90

Run 1 was LoRA with $\alpha = 64$ and $r = 32$ for 200 steps on MATH-12k, with AdamW at $lr = 2 \cdot 10^{-5}$. MATH-12k is close in distribution to MATH-500 without being copies of it (we filtered out prompts too similar to MATH-500). We labelled it with the RLAIF model's pass8 and did difficulty scheduling, exposing the model to easier problems early and harder problems later.

Performance noticeably increased but levelled off around step 125, when almost all groups were all 0s or all 1s (no gradient signal). Thinking this was a data issue, we mixed in more data (DeepMath, DAPO, DeepScaleR), re-labelled prompt difficulties every 50 steps, and ran 100 more steps on top of the 200-step checkpoint ("Run 2").

Zero-variance groups dropped but performance didn't move, so this wasn't a data issue. We then tried a full-parameter fine-tune at $lr = 2 \cdot 10^{-6}$ for 100 steps from the run 1 checkpoint ("Run 3"): this improved math by 3 points, monotonically over checkpoints, so it is not noise. Surprisingly, persona evals stayed strong throughout. Our persona eval is the one from Checkpoint 2: the P_i judge deciding which reply is more Sheldon-like, between the RLAIF seed and the current checkpoint. Even so, we could not match the base model, although basically all of the math lost since the RLAIF seed was recovered.

Finally, we combined the RLAIF and RLVR rewards into one and trained for 80 steps at $lr = 2 \cdot 10^{-6}$, with 8 math and 8 persona prompts per step ("Run 4"). This is the only step where the AI judge got "gamed": the model kept its persona score by writing very short responses, which avoid the RulePenalty and BatchTax docks and earn a higher F_i, since we reward terseness when the user asks for it. The per-term reward trace makes the mechanism clear: P_i compares siblings, so its group mean is 0.5 by construction and it can only re-order responses, whereas the rule and batch-tax terms consistently favor the shorter, plainer sibling every step; the whole reward gain over the run was the rule penalty falling from 0.90 to 0.13 while P_i stayed at 0.50. But a large part of Sheldon's personality is his incessant nerdy rambling, which disappears when the model is terse. Our final model is almost as good on the persona eval and still good at math, but it definitely sounds less Sheldon-like.

Besides this, there were shockingly few tradeoffs between persona and math in the RLVR stage. We think this is because RL is distribution sharpening: RLVR sharpened the distribution we already had from SFT and RLAIF, which has the persona baked in, so you cannot "escape" it via RLVR. The paper "SFT Conflicts, RL Coexists" also shows that RL can keep very different capabilities strong at once, and explains the massive persona/math tradeoff during SFT, where such different tasks clash. The two did not reinforce each other either, which makes sense: Sheldon's rambles are not sound reasoning. So persona and math conflicted heavily during SFT and stayed roughly independent during RL. What remains uncertain is whether the persona judge really captures Sheldonness, whether run 3's 3-point improvement is noise, and whether an earlier run 4 checkpoint would have been the best model, since it takes time to reward-hack into the local minimum the final model ended up in.